2025/05/13/AI_R&D_Center

ナップサック問題における一次元DPによるメモリ削減

23B1014　旦保惟果

やりたいことを、できることに。
湘南工科大学
SHONAN INSTITUTE OF TECHNOLOGY
AI R&D CENTER

目次

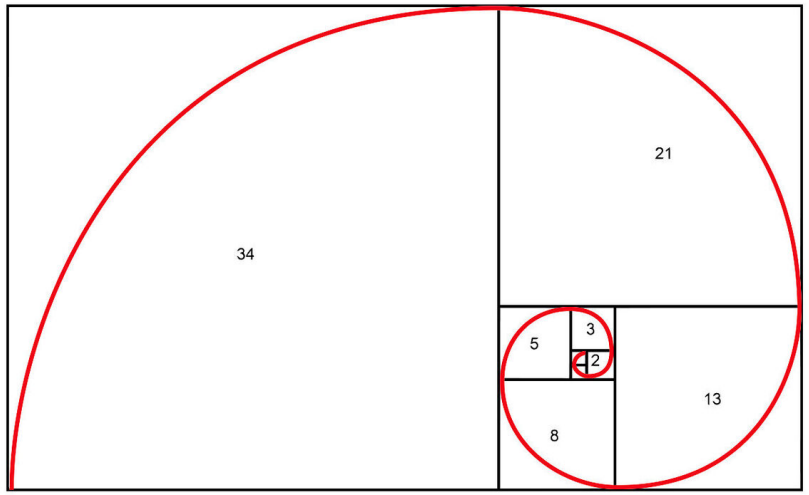
- 確定してから書きます
-
-
-

動的計画法　とは

- **動的計画法**(Dynamic Programming, 以下DP)は, 大きな問題を小さな部分問題に分割して解く手法

- 一度計算した答えをメモしておくことで, 何度も同じ計算をしなくて済む
  ⇒計算量が少なくすむ

- 例
  - グラフの最短経路問題
  - 日本語形態素解析
  - 部分和問題・ナップサック
  - フィボナッチ数列



フィボナッチ数列

- フィボナッチ数列・・・前2つの数を足していくような並びの数列
  - $1,1,2,3,5,8,13,\ldots$

- 第n項を計算するには, 第$n-1$項, 第$n-2$項を求める
  第$n-1$項を計算するには, 第$n-2$項, 第$n-3$項を求める
  $\vdots$
  第2項を計算するには, 第0項と第1項を求める

- 上の計算を下から順番に解いていくことで, 何度も同じ計算をすることなく高速に解が求められる

0-1ナップサック問題

- 品物i 個
  - 価値v_i
  - 重さw_i
- 容量Wのナップサック

- このとき, ナップサックに入る範囲での価値の総和の最大値はいくつ？

二次元DPを用いた基本解放

- 今回のナップサック問題での状態は2つ
  - どこまでの品物を見たか（i個目まで）
  - 今のカバンの重さはどのくらいか（j kg）

- 品物とカバンの重さの2つを小さいものから順番に決定して答えを求める

- $dp[i][j] \coloneqq$ i番目までの品物を考えて重さjになるときの価値の総和の最大値

- この式をもとにして品物iを選ぶ場合と選ばない場合の2つの場合を考える

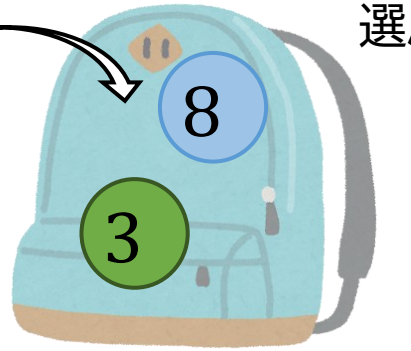
i番目の品物を選ぶ場合

- 選ぶ場合($w_i + j \leq W$)
  - $dp[i][j + w_i] = \max \begin{cases} dp[i-1][j + w_i] \text{（品物}i\text{を選ばないで重さ}j + w_i\text{）} \\ dp[i-1][j] + v_i \text{（品物}i\text{を選んで重さ}j + w_i\text{）} \end{cases}$ を比較

i番目の品物を選ぶ場合

重さ:$j + w_i$
価値:$i - 1$番目までの品物を適切に選んで価値最大

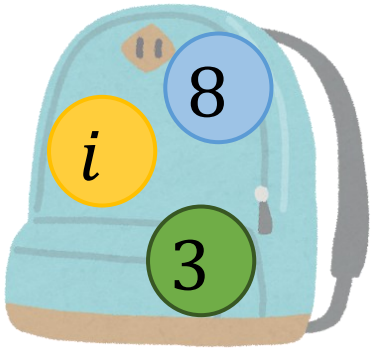
重さ:j
価値:$i - 1$番目までの品物を適切に選んで価値最大



重さ:w_i
価値:v_i

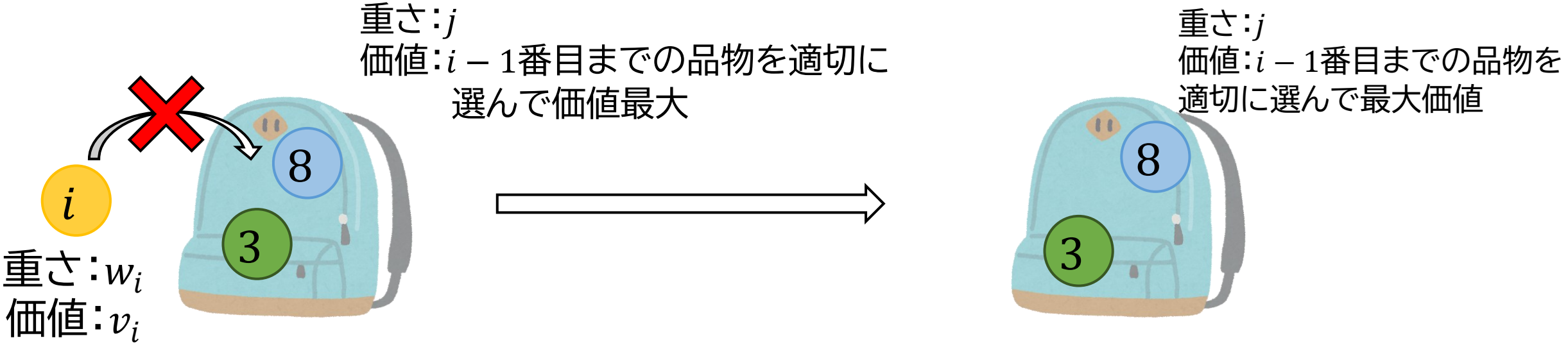
大きい方をとる

重さ:$j + w_i$
価値:($i - 1$番目までの品物を適切に選んで最大価値)$+v_i$



i番目の品物を選ばない場合

- 選ばない場合
  - $dp[i][j] = \begin{cases} dp[i-1][j]\text{（品物}i\text{を選ばない場合）} \\ \mathrm{d}p[i-1][j]\text{（暫定の最大価値）} \end{cases}$ を比較して最大をとる



2次元DPの問題点

- 配列サイズが$N \times W$になる⇒メモリ消費が大きい
- $dp[i][j + w_i]$の更新に使っているのは$dp[i-1][j]$と$dp[i-1][j + w_i]$のみ
- i番目を処理するために, $i-1$番目や$i-2$番目の情報は不要
- 一次元配列だけで十分で, 更新の際に上書きしても支障ない

一次元DPでの解法

- $dp[j]$:=重さjのときの価値の総和の最大値

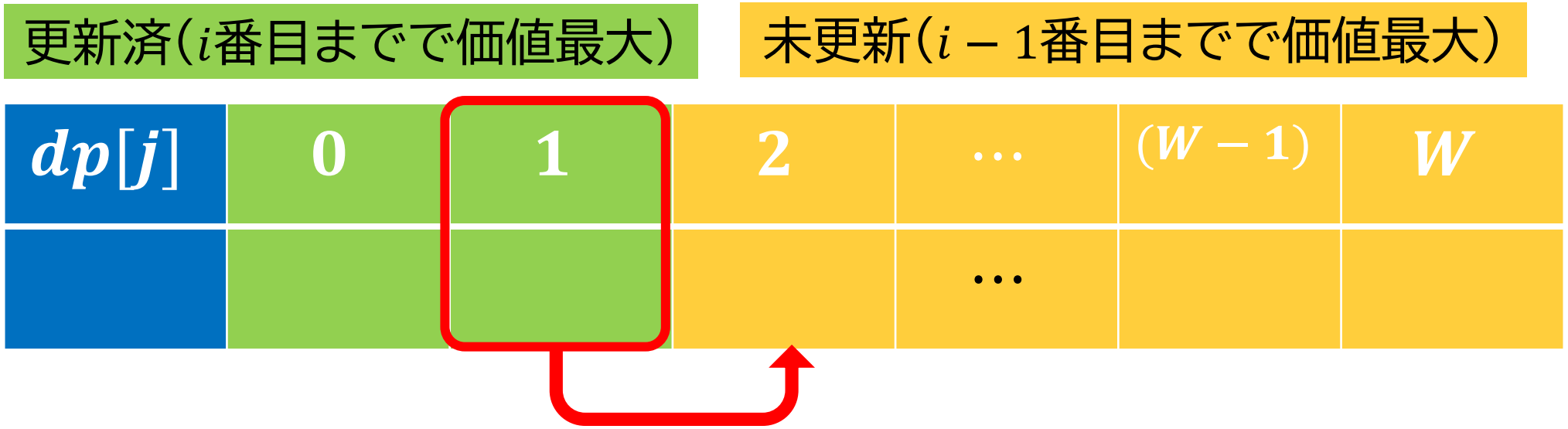
- 品物iを順番に見て，重さjをWからW_iまで逆順に更新

- $dp[j+w_i] = max\begin{cases} dp[j+w_i] \\ dp[j]+v_i \end{cases}$

- このときjを逆順にループしないと，同じ品物を複数回使ってしまう可能性がある

なぜjを逆順に回す必要があるのか

- 順に更新していく



- 「i 番目の品物で重さj になるもの」から「i番目の品物で重さ$j+\alpha$になるもの」の遷移を考えてしまっている

なぜjを逆順に回す必要があるのか

- 逆順に更新していく

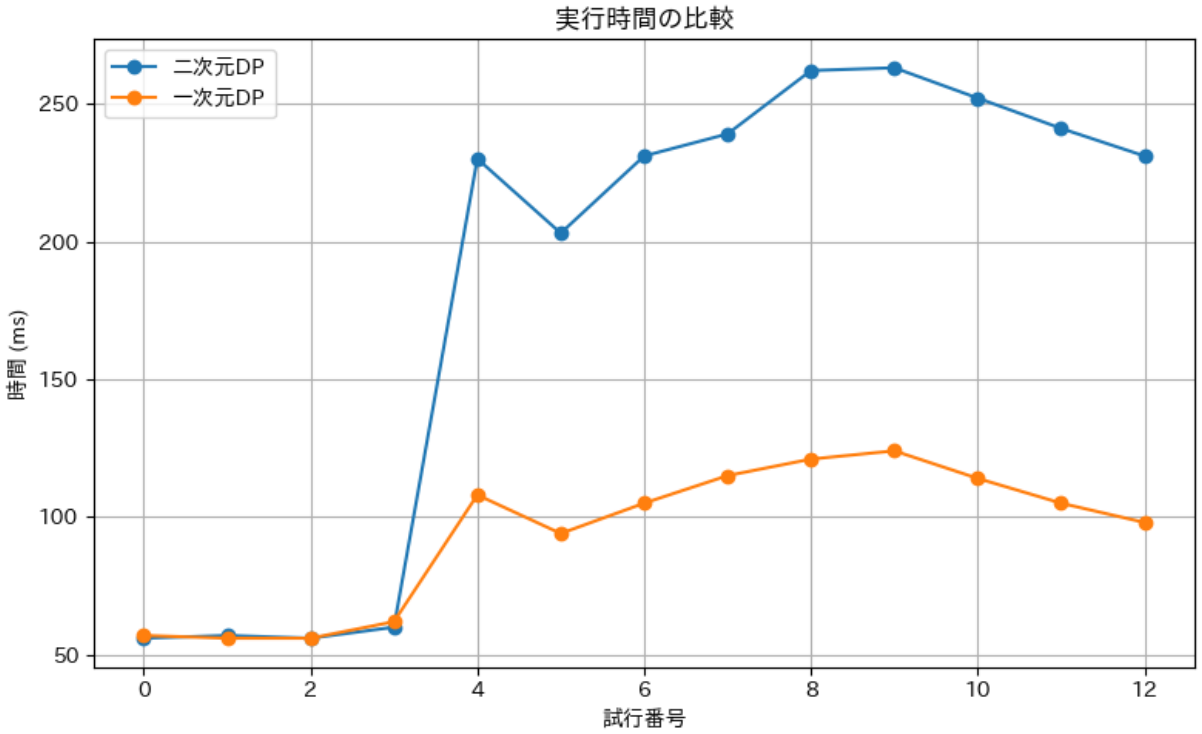
未更新($i-1$番目までで価値最大)　更新済(i番目までで価値最大)

$dp[j]$	0	1	2	…	$(W-1)$	W
				…		

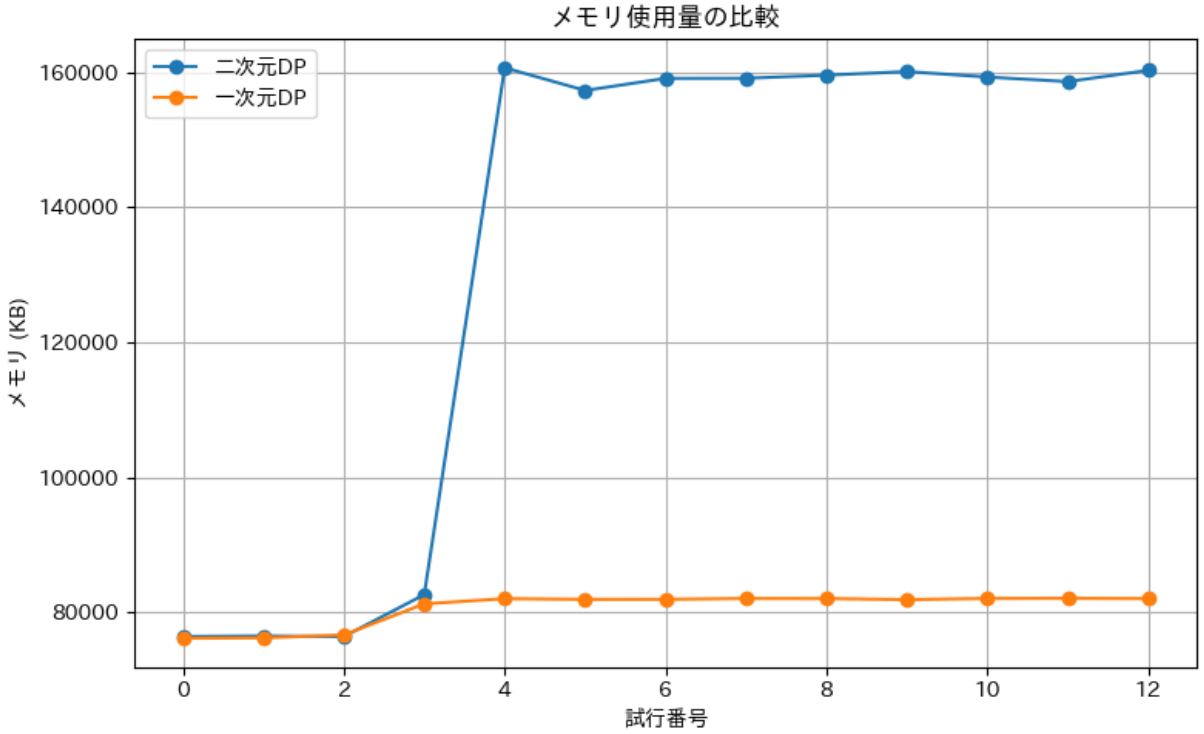
- 「$i-1$番目の品物で重さjになるもの」から「i番目の品物で重さjになるもの」の遷移で考えている
→二次元DPのときの同じ遷移ができている

実行結果

実行時間の比較



メモリの比較



一次元DPのほうが明らかに早いことがわかる

実行結果(AtCoder DPコンテスト Knapsack1より)

- 二次元DP

実行時間	メモリ
56 ms	76412 KB
57 ms	76496 KB
56 ms	76396 KB
60 ms	82624 KB
230 ms	160696 KB
203 ms	157340 KB
231 ms	159128 KB
239 ms	159144 KB
262 ms	159604 KB
263 ms	160140 KB
252 ms	159324 KB
241 ms	158652 KB
231 ms	160320 KB

- 一次元DP

実行時間	メモリ
57 ms	76160 KB
56 ms	76216 KB
56 ms	76640 KB
62 ms	81256 KB
108 ms	82024 KB
94 ms	81924 KB
105 ms	81928 KB
115 ms	82052 KB
121 ms	82040 KB
124 ms	81884 KB
114 ms	82056 KB
105 ms	82076 KB
98 ms	82036 KB

時間計算量　空間計算量とは

- アルゴリズムの性能を比較するための指標の一つ
- 時間計算量
  - 実行に必要な計算ステップ数がどのように変化するか
    （計算ステップ数・・・四則演算, 数値の比較などの回数）
- 空間計算量
  - 実行に必要なメモリの量がどのように変化するか

- つまり
  時間計算量が小さい　→　計算時間が短い
  空間計算量が小さい　→　必要なメモリが少ない
- 計算量が小さければ小さいほど, 良いアルゴリズムだと思ってもらえればいい

まとめ

- 二次元⇒一次元にすることで, 空間計算量が$O(W)$に削減できることで,
  - 大規模な問題サイズにも対応できる
  - 限られたメモリ環境の中でも実装が可能
  - キャッシュ効率が良くなり, メモリアクセス量も減ることで実行時間の向上につながる

今後